

API Gateways & Service Discovery

Phase 2 · Intermediate · Estimated time: 1.5 hours

1. Concept From First Principles

Chapter 11 split a monolith into many independent services. This immediately creates two very practical problems that this chapter solves. First: if a mobile app needs data from the orders service, the payments service, and the user service to render one screen, should it really make three separate network calls to three different backend addresses, and handle three different auth checks? Second: with services being deployed independently, scaled independently, and potentially having multiple running instances whose IP addresses change constantly, how does any one service (or the client) actually find *where* a given service currently lives?

An **API gateway** answers the first problem: it's a single entry point that sits in front of your entire microservices fleet, and every external client (mobile app, web frontend, third-party integrator) talks only to the gateway, never directly to individual services. The gateway's core job is **request routing** — inspecting an incoming request and forwarding it to the correct backend service — but in practice it also centralizes a whole set of cross-cutting concerns that would otherwise need to be duplicated in every single service: authentication and authorization, rate limiting (Chapter 10), request/response logging, and sometimes response aggregation (combining results from multiple backend calls into one response for the client, sparing the client from making multiple round trips itself).

Backend engineer analogy

If you've ever put an Nginx reverse proxy in front of several Node.js or Laravel services running on different ports, routing `/api/orders/*` to one service and `/api/payments/*` to another, you've built a rudimentary API gateway. Production API gateways (Kong, AWS API Gateway, Apigee) do the same routing job but add authentication, rate limiting, and observability as first-class, centrally configured features instead of something you'd have to hand-roll in Nginx config.

Service discovery

The second problem — finding where a service instance currently lives — is **service discovery's** job. In a modern containerized/cloud deployment, service instances are ephemeral: they're created, destroyed, and rescheduled onto different machines constantly (a deploy, an autoscaling event, a crashed instance being replaced), each time potentially getting a new IP address. Hard-coding IP addresses anywhere in this world breaks almost immediately. Service discovery solves this with a **service registry** — a central, continuously updated directory of "which instances of which service are currently healthy and where." When a new instance of, say, the payments service starts up, it **registers** itself with the registry (announcing its address and a health-check endpoint); when another service needs to call payments, it **queries** the registry to get a current, healthy address to call.

There are two common patterns for how this lookup actually happens. In **client-side discovery**, the calling service itself queries the registry directly and picks an instance (often with its own load-balancing logic), which is fast (no extra network hop) but requires every calling service to implement

registry-lookup logic. In **server-side discovery**, the caller simply sends its request to a well-known address (often a load balancer), which itself queries the registry and forwards the request — simpler for calling services, at the cost of an extra network hop through the load balancer. Tools like Netflix Eureka, HashiCorp Consul, and Kubernetes' built-in DNS-based service discovery are the standard implementations you'll encounter in real systems, with Kubernetes' approach being effectively server-side discovery baked directly into the platform.

It's worth being precise about how API gateways and service discovery relate, since interviewers sometimes probe this boundary. The API gateway handles the *external*, client-facing entry point and cross-cutting concerns; service discovery handles the *internal* problem of services finding each other (including the gateway finding the right backend instance to route to). A gateway typically uses service discovery internally to know where to route a request — they're complementary layers solving adjacent but distinct problems, not competing solutions to the same problem.

Health checking deserves a direct mention, since it's what keeps service discovery trustworthy: the registry needs to continuously verify that registered instances are actually alive and healthy (typically via periodic HTTP health-check pings), automatically removing instances that stop responding, so that traffic never gets routed to a dead or unhealthy instance just because it hadn't formally deregistered. This connects directly back to Phase 1's load-balancing chapter and Chapter 2's failover discussion — service discovery is, in a real sense, load balancing's dynamic, distributed-systems-aware cousin.

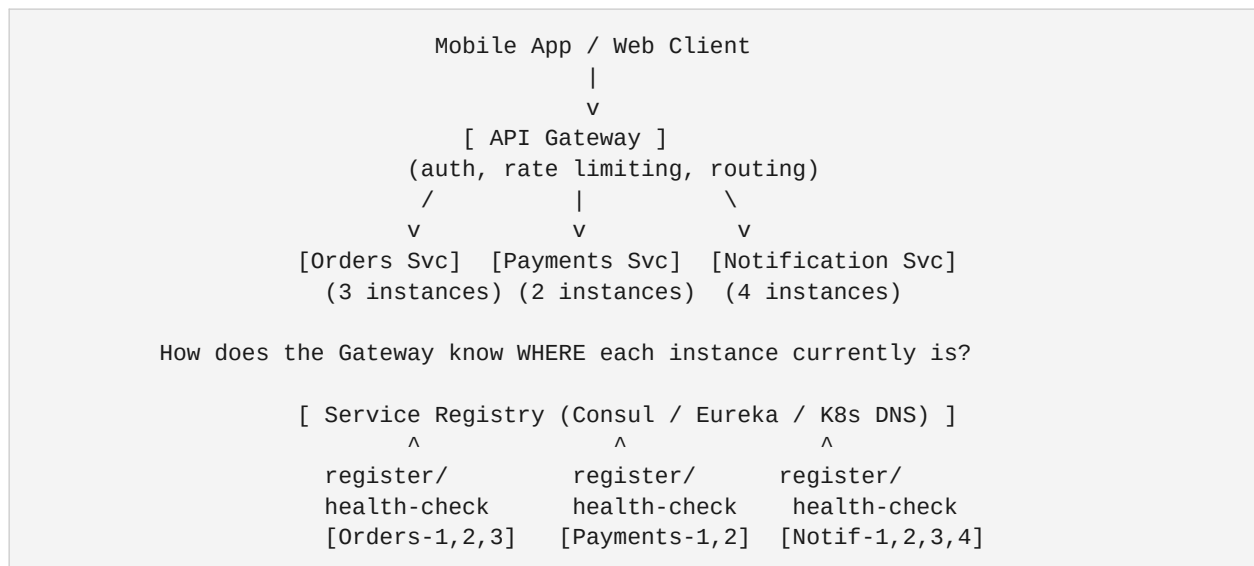
2. Real-World Example / Case Study

Swiggy's mobile app, on opening the home screen, likely needs restaurant listings, active promotions, and the user's order history — all served by different backend services. Rather than the app making three separate calls to three separate backend addresses (and handling three separate auth flows), it makes one call to Swiggy's API gateway, which routes to (and potentially aggregates responses from) the relevant backend services, giving the mobile client a simpler, faster, single integration point.

Razorpay's public developer-facing payment API is itself effectively an API gateway experience for external merchants: merchants integrate against one stable, documented API surface, while behind that gateway, requests get routed to whichever internal services (payment processing, fraud checks, settlement recording) actually handle each piece — merchants never need to know Razorpay's internal service topology, or that it might change entirely without their integration ever needing to change.

A Careem- or Talabat-scale system running on Kubernetes gets service discovery essentially for free through Kubernetes' built-in DNS-based service resolution: a pod (container instance) for the trip-matching service can call **http://pricing-service** without ever knowing or caring about the actual current IP addresses of the pricing service's running instances, which Kubernetes handles transparently and keeps current as instances scale up, scale down, or get rescheduled.

3. Diagram



4. Trade-offs Table

Component	Solves	Cost if skipped
API Gateway	Single entry point, centralized auth/rate limiting/routing	Every client integrates with N services separately; duplicated cross-cutting logic
Service Registry	Dynamic, current directory of healthy service instances	Hard-coded IPs break on every deploy/restart/autoscale
Client-side discovery	Fast, no extra hop	Every service must implement registry-lookup + load-balancing logic
Server-side discovery	Simple for calling services	Extra network hop through a load balancer/gateway

5. Common Interview Questions

Q1: "Why put an API gateway in front of your microservices instead of letting clients call them directly?"

Model approach: Name the concrete costs of direct client-to-service calls — duplicated auth logic in every service, clients needing to know internal service topology, no central place for rate limiting or logging — and explain how a gateway centralizes all of these into one well-tested layer.

Q2: "How does Service A find the current address of Service B in a system with dozens of dynamically-scaled instances?"

Model approach: Describe the register/query flow through a service registry, name a real tool (Consul, Eureka, or Kubernetes DNS), and explain the role of health checks in keeping the registry accurate as instances come and go.

Q3: "What's the difference between client-side and server-side service discovery?"

Model approach: Client-side means the calling service queries the registry itself and picks an instance directly; server-side means the caller hits a well-known address (a load balancer) that does the registry lookup on its behalf. Trade fewer network hops (client-side) against simpler calling-service code (server-side).

Q4: "Can an API gateway become a single point of failure? How do you mitigate that?"

Model approach: Yes — acknowledge it directly, then apply Chapter 2's redundancy principles: run multiple gateway instances behind their own load balancer, so the gateway layer itself has no single point of failure, the same way any other critical component needs redundancy.

6. Practice Exercises

- 1 Design an API gateway's responsibilities for a food delivery app's mobile client, listing at least four cross-cutting concerns it should centralize.
- 2 Explain what breaks in a Kubernetes-free, manually-deployed microservices system if you hard-code each service's IP address in every other service's configuration.
- 3 Compare client-side vs. server-side service discovery for a high-throughput internal service-to-service call path, and justify which you'd choose.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"System Design - Part 19 API Gateway An overview, explained with diagrams and flow"	Clear diagram-driven walkthrough of API gateway responsibilities.
English	"API Gateway Microservices Design Pattern Swiggy Architecture (Imaginary)" — Tech Primers	Applies the API gateway pattern to a familiar, India-based example architecture.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers API gateways and service discovery as part of a full bilingual course.
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp	Places this chapter's topics in context of the broader Hindi HLD roadmap.

8. Key Takeaways

- An API gateway is the single entry point for external clients into a microservices architecture.
- Gateways centralize routing, authentication, rate limiting, and logging so services don't duplicate them.
- Service discovery solves the problem of dynamically finding healthy service instances as they scale and restart.
- A service registry tracks which instances are alive via registration and continuous health checks.
- Client-side discovery trades an extra network hop for calling-service simplicity; server-side does the opposite.
- API gateways and service discovery are complementary layers, not competing solutions to the same problem.

- The gateway itself needs redundancy (Chapter 2) to avoid becoming a new single point of failure.